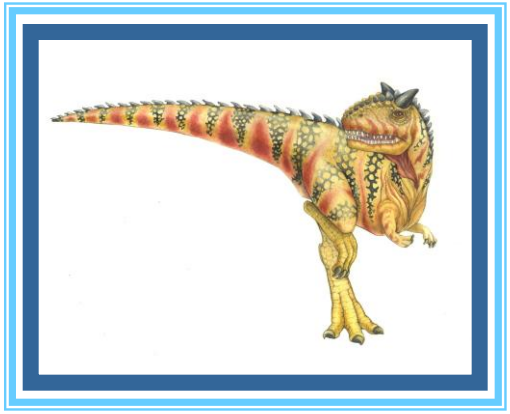
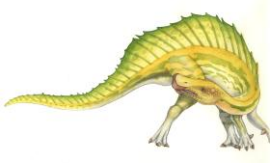
Chapter 3: Processes



Outline

- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- IPC in Shared-Memory Systems
- IPC in Message-Passing Systems
- Examples of IPC Systems
- Communication in Client-Server Systems



Objectives

- Identify the separate components of a process and illustrate how they are represented and scheduled in an operating system.
- Describe how processes are created and terminated in an operating system, including developing programs using the appropriate system calls that perform these operations.
- Describe and contrast interprocess communication using shared memory and message passing.
- Design programs that uses pipes and POSIX shared memory to perform interprocess communication.
- Describe client-server communication using sockets and remote procedure calls.
- Design kernel modules that interact with the Linux operating system.

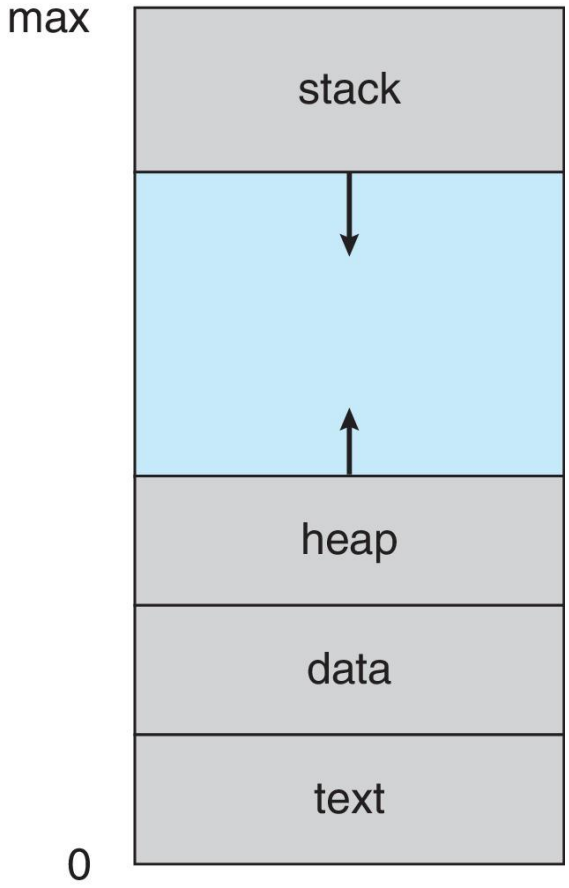
Process Concept

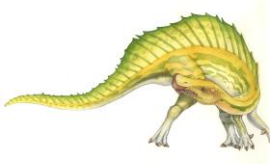
- An operating system executes a variety of programs that run as a process.
- **Process** – a program in execution; process execution must progress in sequential fashion. No parallel execution of instructions of a single process
- Multiple parts
  - The program code, also called **text section**
  - Current activity including **program counter**, processor registers
  - **Stack** containing temporary data
    - Function parameters, return addresses, local variables
  - **Data section** containing global variables
  - **Heap** containing memory dynamically allocated during run time

Process Concept (Cont.)

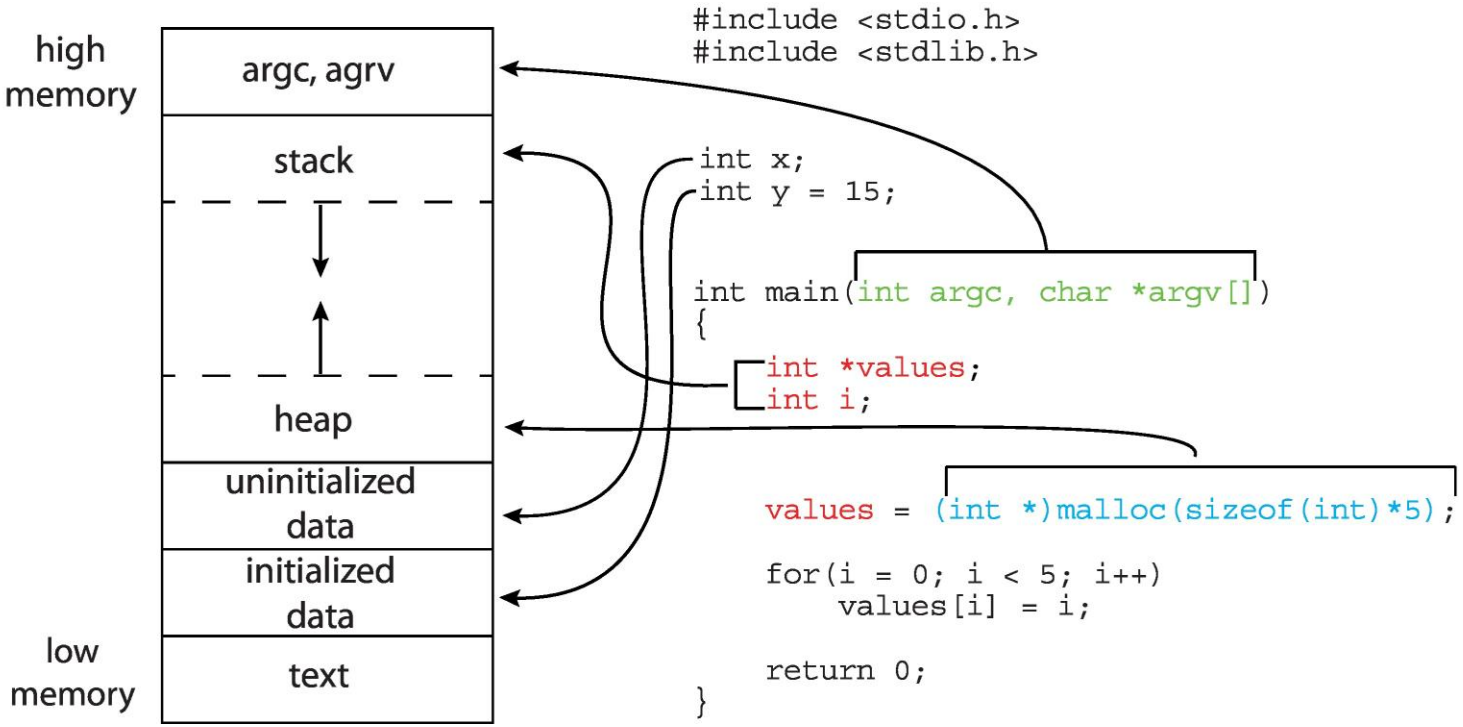
- Program is **passive** entity stored on disk (**executable file**); process is **active**
  - Program becomes process when an executable file is loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc.
- One program can be several processes
  - Consider multiple users executing the same program

Process in Memory





Memory Layout of a C Program



Process State

1. New State

- A process is said to be in the **New** state when a program present in the secondary memory is initiated for execution.

2. Ready State

- A process moves from the **New** state to the **Ready** state after it is loaded into the main memory and is ready for execution.
- In the **Ready** state, the process waits for its execution by the processor.
- In a multiprogramming environment, many processes may be present in the **Ready** state.

3. Run State

- A process moves from the **Ready** state to the **Run** state after it is assigned the CPU for execution.

4. Terminate State

- A process moves from the **Run** state to the **Terminate** state after its execution is completed.
- After entering the **Terminate** state, the context (**PCB**) of the process is deleted by the operating system.

Process State (Cont.)

5. Block or Wait State

- A process moves from the **Run** state to the **Block (Wait)** state if it requires an I/O operation or some blocked resource during its execution.
- After the I/O operation is completed or the required resource becomes available, the process moves to the **Ready** state.

6. Suspend Ready State

- A process moves from the **Ready** state to the **Suspend Ready** state if a higher-priority process has to be executed but the main memory is full.
- Moving a lower-priority process from the **Ready** state to the **Suspend Ready** state creates room for the higher-priority process in the **Ready** state.
- The process remains in the **Suspend Ready** state until the main memory becomes available.
- When main memory becomes available, the process is brought back to the **Ready** state.

7. Suspend Wait State

- A process moves from the **Wait** state to the **Suspend Wait** state if a higher-priority process has to be executed but the main memory is full.

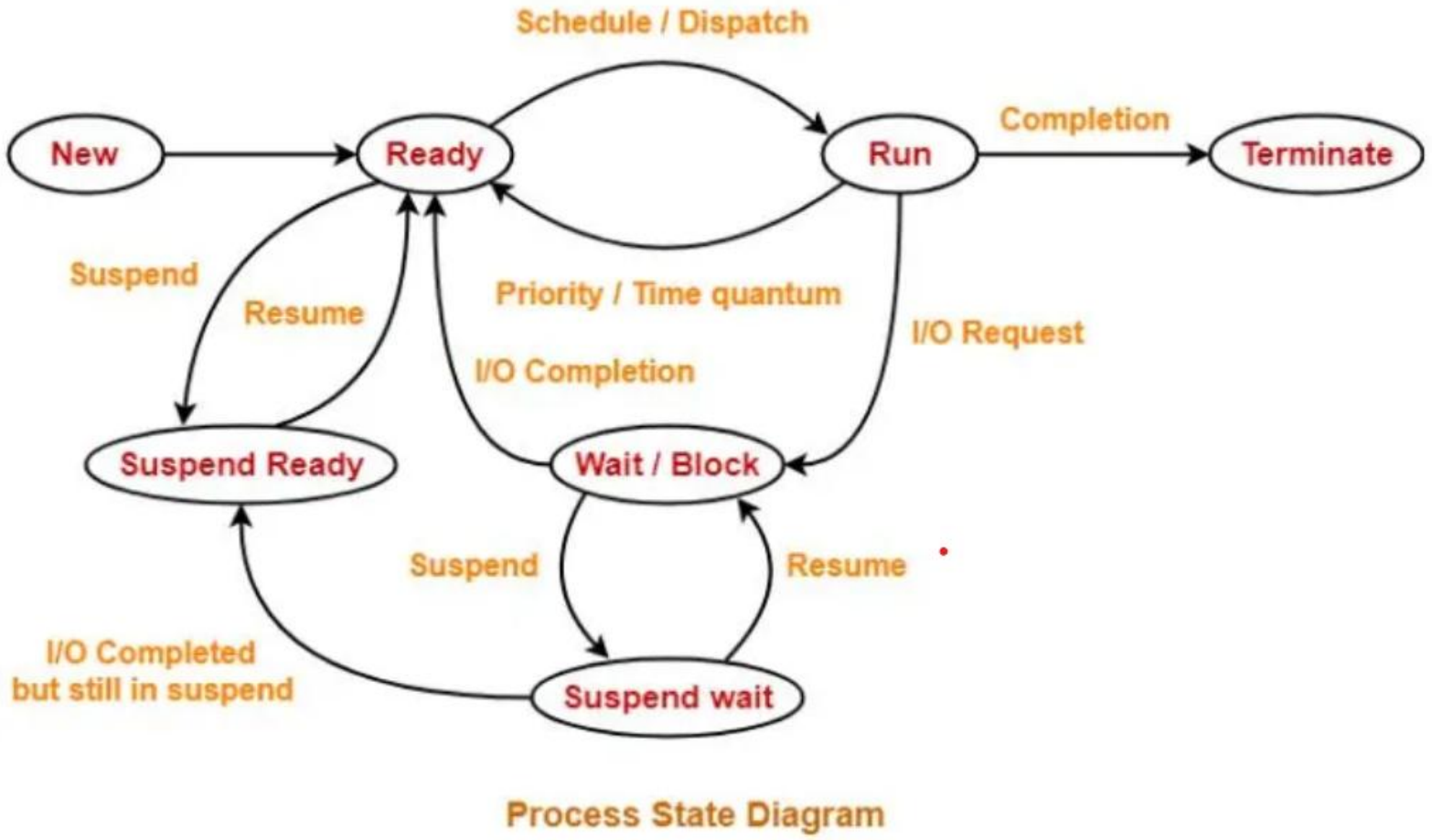
Process State (Cont.)

7. Suspend Wait State (Continued)

- Moving a lower-priority process from the **Wait** state to the **Suspend Wait** state creates room for a higher-priority process in the **Ready** state.
- After the required resource becomes available, the process is moved to the **Suspend Ready** state.
- After the main memory becomes available, the process is moved to the **Ready** state.

State	Present in Memory
New state	Secondary Memory
Ready state	Main Memory
Run state	Main Memory
Wait state	Main Memory
Suspend wait state	Secondary Memory
Suspend ready state	Secondary Memory
Terminate state	–

Diagram of Process State



Process State Diagram

Process State

- As a process executes, it changes **state**
  - **New**: The process is being created
  - **Ready**: The process is waiting to be assigned to a processor
  - **Running**: Instructions are being executed
  - **Waiting**: The process is waiting for some event to occur
  - **Terminated**: The process has finished execution

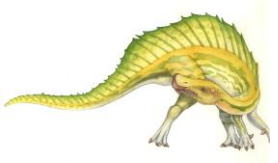
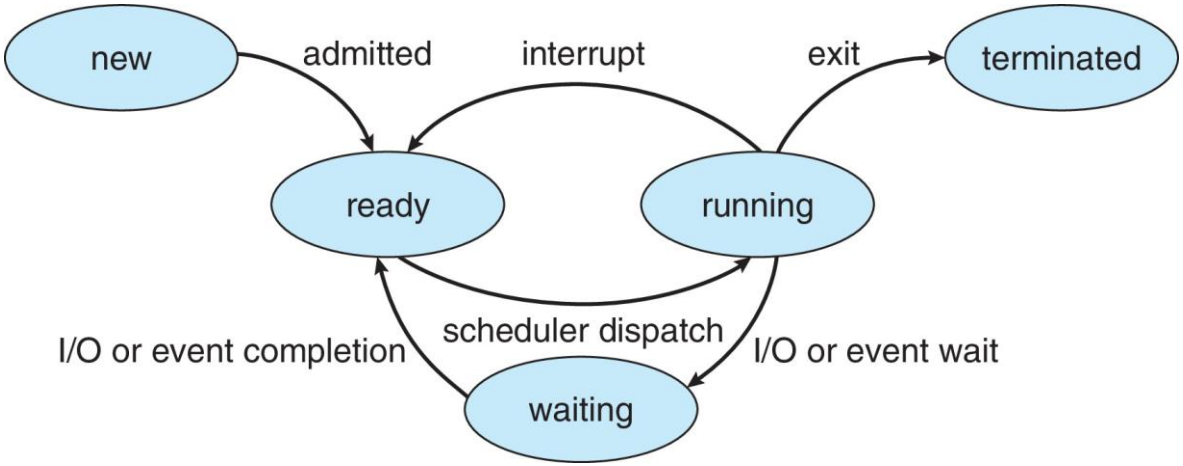


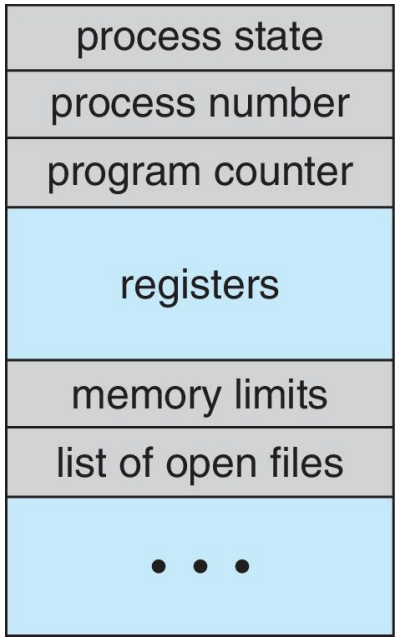
Diagram of Process State

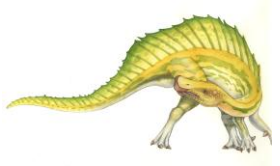


Process Control Block (PCB)

Information associated with each process(also called **task control block**)

- Process ID – Every process has unique number to differentiate one process from other.
- Process state – running, waiting, etc.
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files





Operations on the Process

1. Creation

- Once the process is created, it enters the **Ready Queue (main memory)** and becomes ready for execution.

2. Scheduling

- Out of the many processes present in the **Ready Queue**, the operating system selects one process for execution.
- The process of selecting the next process to be executed is known as **Scheduling**.

3. Execution

- Once the process is scheduled, the processor starts executing it.
- During execution, the process may enter the **Blocked (Wait)** state if it requires an I/O operation or another resource. In that case, the processor begins executing another ready process.

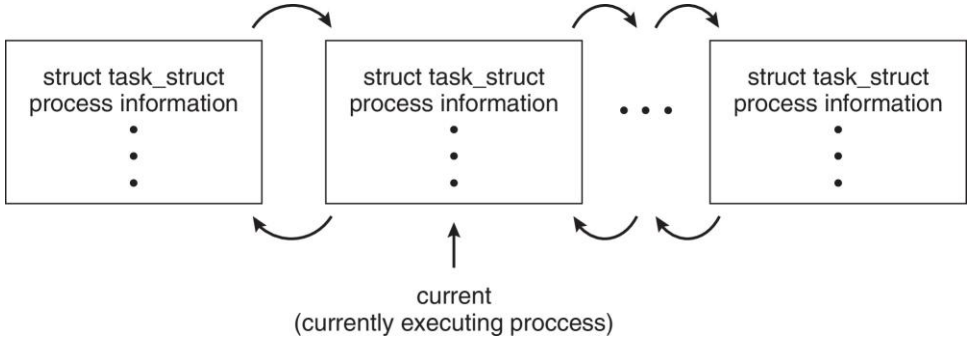
4. Deletion (Killing)

- Once the purpose of the process is completed, the operating system terminates (kills) the process.
- The **Process Control Block (PCB)** of the process is deleted, and the process is removed from the system.

Process Representation in Linux

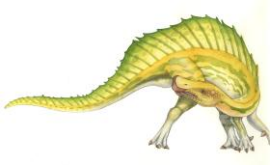
Represented by the C structure `task_struct`

```
pid_t pid;                    /* process identifier */
long state;                   /* state of the process */
unsigned int time_slice       /* scheduling information */
struct task_struct *parent;/* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files;/* list of open files */
struct mm_struct *mm;         /* address space of this
process */
```



Process Scheduling

- **Process scheduler** selects among available processes for next execution on CPU core
- Goal -- Maximize CPU use, quickly switch processes onto CPU core
- Maintains **scheduling queues** of processes
  - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
  - **Wait queues** – set of processes waiting for an event (i.e., I/O)
  - Processes migrate among the various queues



Types of Schedulers

- Long term scheduler
  - selects process and loads it into ready queue (memory) for execution
- Medium term scheduler
  - Memory manager
  - Swap in, Swap out from main memory non-active processes
- Short term scheduler
  - Deals with processes among ready processes
  - Very fast, similar to action at every clock interrupt
  - if a process requires a resource (or input) that it does not have, it is removed from the ready list (and enters the WAITING state)

Long-Term Schedulers

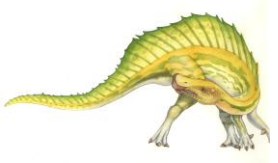
- Long-term scheduler also called job scheduler.
- The long-term scheduler controls the degree of multiprogramming (the number of processes in memory).
- In multiprogramming system, more processes are submitted than can be executed immediately.
- These processes are spooled to a mass-storage device (typically a disk), where they are kept for later execution.
- Job scheduler selects processes from this pool and loads them into memory for execution.
- The long-term scheduler executes much less frequently.
- It is important that the long-term scheduler make a careful selection of both IO and CPU bound process.
- IO bound tasks are which use much of their time in input and output operations
- while CPU bound processes are which spend their time on CPU.
- The job scheduler increases efficiency by maintaining a balance between the IO and CPU bound process.

Short-Term Schedulers / CPU Schedulers

- Short-term scheduler also called CPU scheduler.
- The role of the CPU scheduler is to select from among the processes that are in the ready queue and allocate a CPU core to one of them.
- The CPU scheduler is responsible for ensuring there is no starvation owing to high burst time processes.
- Short-term scheduler only selects the process to schedule it doesn't load the process on running.
- Dispatcher is responsible for loading the process selected by Short-term scheduler on the CPU.
- Short-term schedulers are faster than long-term schedulers.

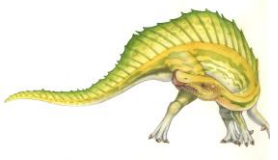
Short-Term Schedulers

- The **CPU scheduler** selects from among the processes in ready queue, and allocates a CPU core to one of them
  - Queue may be ordered in various ways
- CPU scheduling decisions may take place when a process:
  1. Switches from running to waiting state
  2. Switches from running to ready state
  3. Switches from waiting to ready
  4. Terminates
- For situations 1 and 4, there is no choice in terms of scheduling. A new process (if one exists in the ready queue) must be selected for execution.
- For situations 2 and 3, however, there is a choice.



Preemptive and Nonpreemptive Scheduling

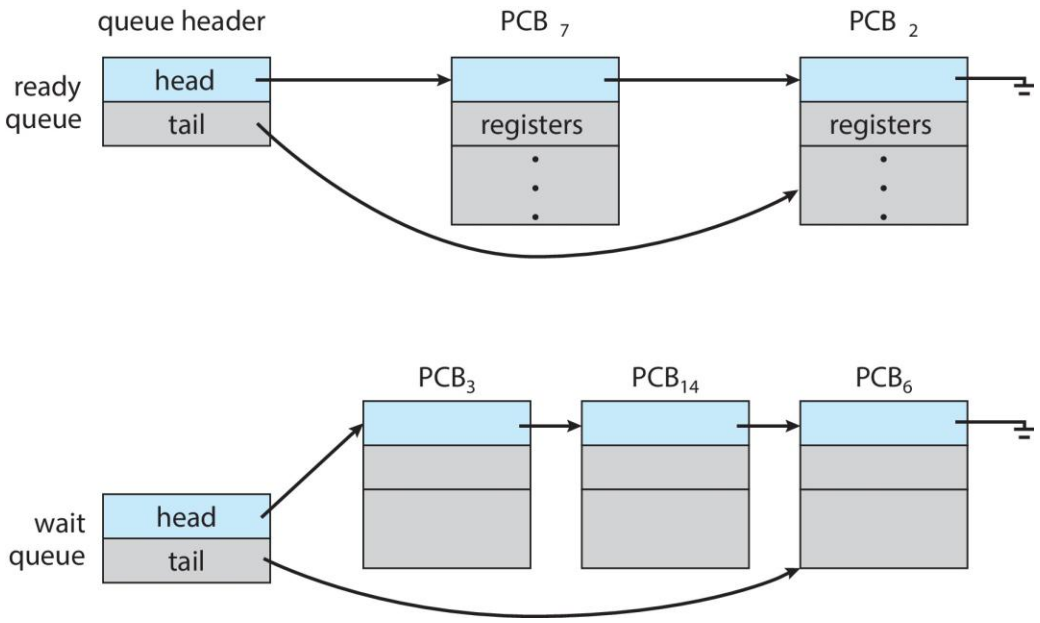
- When scheduling takes place only under circumstances 1 and 4, the scheduling scheme is **nonpreemptive**.
- Otherwise, it is **preemptive**.
- Under Nonpreemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases it either by terminating or by switching to the waiting state.
- Virtually all modern operating systems including Windows, MacOS, Linux, and UNIX use preemptive scheduling algorithms.



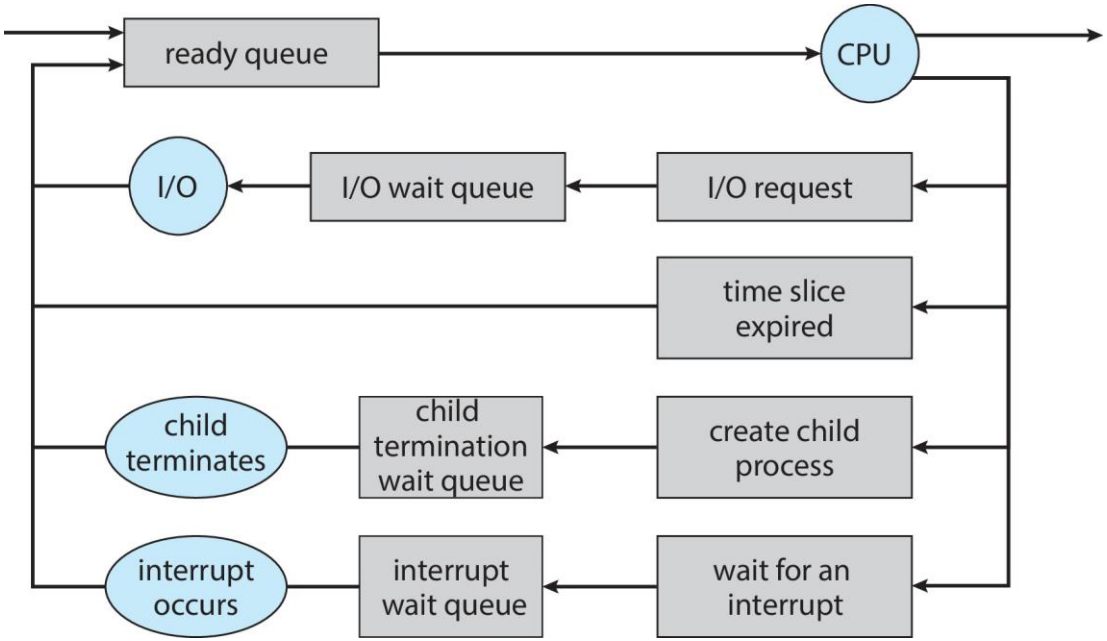
Medium-Term Schedulers

- Some operating systems, such as time-sharing systems, may introduce an additional, intermediate level of scheduling.
- The key idea behind a medium-term scheduler is that sometimes it can be advantageous to remove processes from memory (and from active contention for the CPU) and thus reduce the degree of multiprogramming.
- Later, the process can be reintroduced into memory, and its execution can be continued where it left off. This scheme is called swapping.
- The process is swapped out, and is later swapped in, by the **medium-term scheduler**.
- It is responsible for suspending and resuming the process.
- Swapping may be necessary to improve the process mix or because a change in memory requirements has overcommitted available memory, requiring memory to be freed up.

Ready and Wait Queues



Representation of Process Scheduling

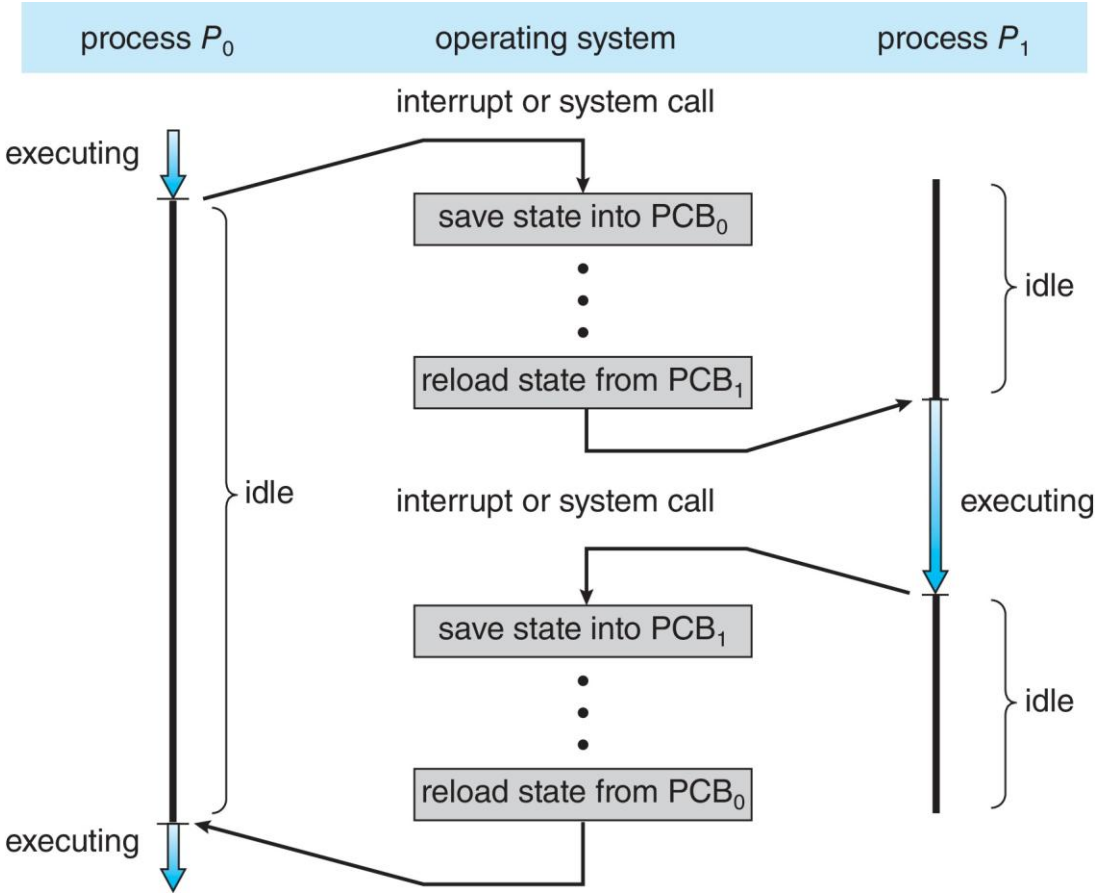


Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is pure overhead; the system does no useful work while switching
  - The more complex the OS and the PCB ➔ the longer the context switch
- Time dependent on hardware support
  - Some hardware provides multiple sets of registers per CPU ➔ multiple contexts loaded at once

CPU Switch From Process to Process

A **context switch** occurs when the CPU switches from one process to another.



Operations on Processes

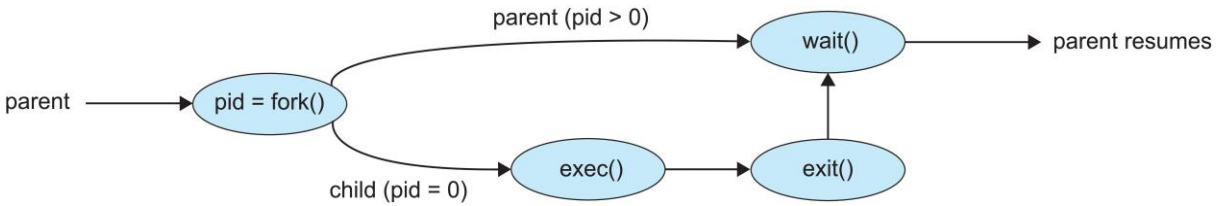
- System must provide mechanisms for:
  - Process creation
  - Process termination

Process Creation

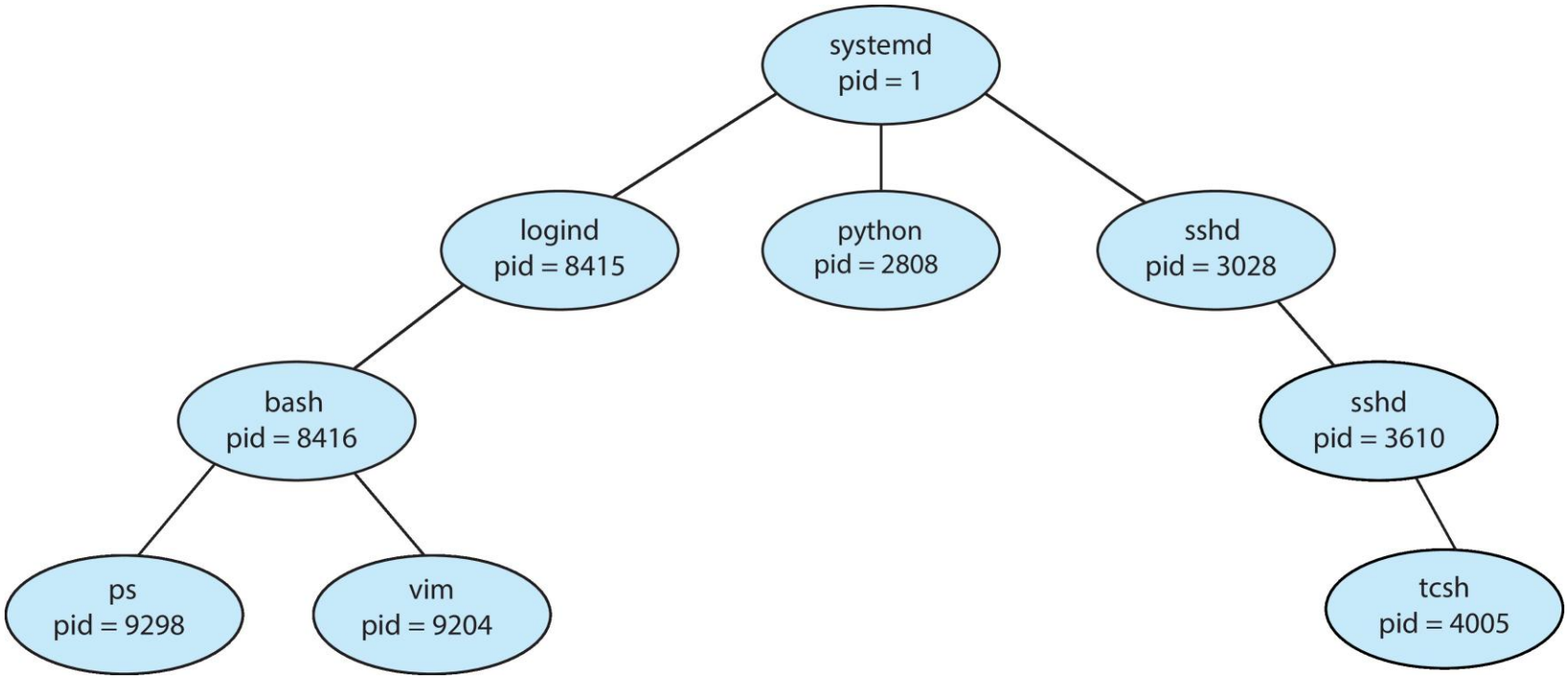
- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, process identified and managed via a **process identifier** (**pid**)
- Resource sharing options
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate

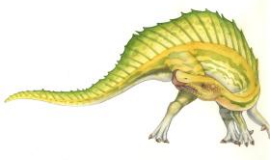
Process Creation (Cont.)

- Address space
  - Child duplicate of parent
  - Child has a program loaded into it
- UNIX examples
  - **`fork()`** system call creates new process
  - **`exec()`** system call used after a **`fork()`** to replace the process' memory space with a new program
  - Parent process calls **`wait()`** waiting for the child to terminate



A Tree of Processes in Linux





C Program Forking Separate Process

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```

Process Termination

- Process executes last statement and then asks the operating system to delete it using the **`exit()`** system call.
  - Returns status data from child to parent (via **`wait()`**)
  - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the **`abort()`** system call. Some reasons for doing so:
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - The parent is exiting, and the operating systems does not allow a child to continue if its parent terminates

Process Termination

- Some operating systems do not allow child to exists if its parent has terminated. If a process terminates, then all its children must also be terminated.
  - **cascading termination.** All children, grandchildren, etc., are terminated.
  - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the **`wait()`** system call. The call returns status information and the pid of the terminated process

```
pid = wait(&status);
```

- If no parent waiting (did not invoke **`wait()`**) process is a **zombie**
- If parent terminated without invoking **`wait()`**, process is an **orphan**

End of Chapter 3